

Multiple root layouts

Live demo: `/` (marketing) and `/cart` (shop) — run this app separately. **Code:** `examples/multiple-root-layouts/src/app/`

What it is

Normally an app has **one** root layout at `app/layout.tsx` that renders `<html>` / `<body>` and wraps everything. To give different sections **completely different shells**, you:

1. **Delete the top-level** `app/layout.tsx` .
2. Create a **route group** per section and put a **root layout inside each**.
3. Because each is now a root layout, **each must render its own** `<html>` **and** `<body>` .

This app's structure

```
src/app/
(marketing)/
  layout.tsx  ← root layout #1 (its own <html>/<body>) → wraps "/"
  page.tsx   ← "/"
(shop)/
  layout.tsx  ← root layout #2 (its own <html>/<body>) → wraps "/cart"
  cart/page.tsx ← "/cart"
```

There is **no** `src/app/layout.tsx` .

Rules & gotchas

- Each root layout **must** include `<html>` and `<body>` (there's no parent to provide them).
- **No two pages may resolve to the same URL** across the groups (e.g. don't put a `page.tsx` in both groups at the root — both would be `/`).
- Every page must be covered by exactly one root layout.

When to use

Partitioning an app into sections with fundamentally different UI/experience — e.g. a marketing site and a logged-in app, or a storefront and an admin panel — served from the same project.

Interview Q&A

- **Q: How do you create multiple root layouts?** Remove the top-level `app/layout.tsx` , then add a `layout.tsx` inside each route group; each renders its own `<html>` / `<body>` .
- **Q: What's the catch with multiple root layouts?** Each must render `<html>` / `<body>` , and no two routes can map to the same URL.